

Dự án nhỏ: Quản lý sinh viên

Áp dụng OOP xây dựng chương trình quản lý sinh viên.

Mục tiêu bài học

- Áp dụng OOP để xây dựng chương trình quản lý sinh viên
- Kết hợp lớp, danh sách, hàm và vòng lặp trong một dự án nhỏ
- Rèn kỹ năng chia chương trình thành các phần nhỏ

1. Yêu cầu bài toán

Xây dựng chương trình console quản lý danh sách sinh viên với các chức năng: thêm sinh viên, hiển thị danh sách, tìm theo tên, xóa sinh viên và thống kê điểm trung bình.

2. Thiết kế

- Lớp `SinhVien` lưu mã, tên, điểm và có phương thức `xep_loai()`.
- Lớp `QuanLySinhVien` chứa danh sách (list) các sinh viên và các phương thức xử lý: `them`, `hien_thi`, `tim_theo_ten`, `xoa`, `diem_trung_binh`.
- Hàm `main()` hiển thị menu, nhận lựa chọn của người dùng bằng vòng lặp `while`.

3. Cài đặt mẫu

```
class SinhVien:
    def __init__(self, ma, ten, diem):
        self.ma, self.ten, self.diem = ma, ten, diem

    def xep_loai(self):
        return "Giỏi" if self.diem >= 8 else "Khá" if self.diem >= 6.5 else "Trung bình"

class QuanLySinhVien:
    def __init__(self):
        self.ds = []

    def them(self, sv):
        self.ds.append(sv)

    def tim_theo_ten(self, tu_khoa):
        return [sv for sv in self.ds if tu_khoa.lower() in sv.ten.lower()]

    def diem_trung_binh(self):
        return sum(sv.diem for sv in self.ds) / len(self.ds) if self.ds else 0
```

4. Gợi ý mở rộng

Lưu danh sách ra file (văn bản hoặc JSON) để dữ liệu không mất khi tắt chương trình; kiểm tra dữ liệu nhập (điểm từ 0 đến 10, mã không trùng); sắp xếp danh sách theo điểm.

Tóm tắt

- Chia bài toán thành các lớp có trách nhiệm rõ ràng.
- Dùng list để lưu tập đối tượng, duyệt bằng vòng lặp hoặc list comprehension.
- Menu bằng `while` giúp chương trình chạy liên tục đến khi người dùng thoát.

Bài tập vận dụng

1. Hoàn thiện chương trình với đầy đủ menu và các chức năng ở trên.
2. Thêm chức năng sắp xếp danh sách theo điểm giảm dần.
3. Thêm chức năng ghi và đọc danh sách sinh viên từ file JSON.